S. Sai Hemanth

Career Objective : Game Dev

Phone : +91 8897696573

Mail : saihemanth.s@outlook.com

Social : linkedin, github, youtube

About me

I'm a game systems engineer and game designer with over five years of experience building high-performance games, XR applications, and real-time simulation systems using Unity. My work focuses on gameplay architecture, rendering optimization, multi-threaded programming, and saleable technical systems that remain maintainable as projects grow.

I enjoy solving complex engineering problems—from GPU rendering and compute shaders to multiplayer architecture, AI systems, and open-world streaming. Whether leading a development team, mentoring junior developers, or prototyping new gameplay mechanics, I approach every project with the mindset of building systems that are robust, performant, and enjoyable to work with.

Beyond programming, I enjoy translating technical ideas into engaging player experiences, combining engineering with game design to create immersive worlds and meaningful interactions.

Core Skills

- **Game Engines** : Unity, Unreal, Cry engine
- **Programming Languages** : C#, C++, Python
- **Platforms** : PC, XR (Meta quest), Android, IOS
- **Gameplay** : Systems, Mechanics, AI, Multiplayer, Editor tools

Featured projects

Leonids - High performance data driven ballistics system

Video available here : Youtube

The Problem :

Traditional projectile implementations in Unity treat every bullet as an individual GameObject containing components such as a Transform, MeshRenderer, Rigidbody, and behaviour scripts. While object pooling reduces allocation overhead, every active projectile must still be updated individually each frame on the main thread, resulting in an O(n) update loop that scales poorly for bullet-heavy games.

For large-scale combat scenarios involving tens or hundreds of thousands of projectiles, the main thread quickly becomes the performance bottleneck.

The Solution :

Instead of treating projectiles as objects, Leonids treats them as data. During the loading screen, the framework preallocates a fixed-size *NativeArray<ProjectileData>* capable of storing up to one million projectiles. Since the maximum projectile count is known beforehand, runtime allocations are completely eliminated.

To avoid performing an O(n) search for free slots, Leonids uses a rolling index allocator, allowing new projectiles to be inserted in constant time (O(1)). Once allocated, projectile simulation is entirely data-driven and executed using Unity's Burst Compiler and Job System across multiple CPU cores.

Each simulation frame consists of five stages:

Projectile Simulation :

1. **Projectile Simulation**
    a) Advance projectile positions.
    b) Update lifetime.
    c) Skip inactive projectiles.
2. **Raycast Preperation**
    a) Generate batched RaycastCommands from the previous and newly simulated positions.
    b) This performs continuous collision detection without requiring Rigidbody physics.
3. **Collision Evaluation**
    a) Execute batched raycasts asynchronously.
    b) Determine hit or miss for each projectile
4. **Result Dispatch**
    a) Write collision results into a thread-safe, multi write supported queue.
5. **Rendering**
    a) Prepare the graphics buffer using bullet position and direction they are moving.
    b) Depending on the target hardware we can use an URP render pass for instanced indirect rendering, or we can use compute shader to render the projectile meshes or we can use VFX graph to render bullet meshes. All these three techniques use the buffers created above to function.

The main thread dequeues events and publishes them through an Event Bus, allowing gameplay systems such as damage, VFX, audio, decals, and AI to react independently without tightly coupling themselves to the projectile framework. Rendering of projectiles doesn’t involve another O(n) to manually set transforms of various game objects, but we use one of the above mentioned techniques in simulation step 5 and consume the graphics buffer directly on the GPU thus saving a lot of compute time on the CPU.

Key Engineering Decisions :

- Data-oriented architecture using NativeArray<ProjectileData>
- Burst-compiled multithreaded simulation
- Rolling index allocator for O(1) projectile allocation
- Batched collision detection using RaycastCommand.ScheduleBatch
- Event-driven gameplay architecture
- Fixed memory footprint with zero runtime allocations
- Supports 1,000,000 simulated projectiles every single frame
- The data is arranged in Structure of Arrays format to be cache friendly on the CPU making this faster by 30% compared Array of Structures implementation.

Hermes - Event driven gameplay messages system (Unity only)

Video available here : Youtube

The Problem :

As projects grow in size, gameplay systems often become tightly coupled through direct object references or singleton managers. While this approach works well for small prototypes, it quickly becomes difficult to maintain as more systems need to react to gameplay events.

For example, when a projectile hits an NPC, the projectile may need to directly reference the NPC's health component, alert nearby AI, update the UI, spawn particle effects, play audio, notify the quest system, and record analytics. Every new feature introduces another dependency, increasing coupling between systems. A missing reference or an incorrectly configured object can easily introduce bugs, making the project harder to extend and maintain over time.

The Solution :

Hermes replaces direct communication with an event-driven publish–subscribe architecture. Instead of interacting directly with other gameplay systems, objects simply broadcast strongly typed events implementing an IEvent interface. Any system interested in that event can subscribe independently and react using the provided payload.

This architecture completely decouples the sender from the receivers. The projectile no longer knows whether AI, UI, audio, VFX, quests, networking, or analytics are listening—it simply publishes a ProjectileHitEvent containing the required gameplay data. Systems register themselves with the event bus and automatically receive only the events they care about.

Hermes has become one of the core architectural systems used across my projects, enabling modular gameplay development where new features can be added by subscribing to existing events rather than modifying existing gameplay code.

Engineering Decisions :

- Generic event publishing
- Strongly typed payloads
- Interface-based event definition (*IEvent*)
- Zero knowledge between sender and receiver
- Publish–Subscribe architecture
- Supports arbitrary payload structures
- Modular gameplay communication

Learning :

Developing Hermes reinforced the importance of software architecture over implementation. I realized that reducing dependencies between systems often has a greater long-term impact than optimizing individual algorithms. By designing gameplay around events rather than direct references, features became significantly easier to maintain, debug, extend, and reuse across multiple projects.

The framework has fundamentally changed how I approach gameplay programming. Today, almost every large system I design—including projectile simulation, AI, UI, interaction systems, and gameplay notifications—is built around event-driven communication rather than tightly coupled object references.

The architectural principles behind Hermes were validated during my professional work at Mad VR Solutions, where the same event-driven approach was successfully used to coordinate communication between multiple gameplay systems in production-scale VR applications.

Unreal Async Asset Streaming Framework

The Problem :

During gameplay prototyping I observed that weapon actors, pickups, and interactable objects often loaded their meshes, materials, textures, physics assets, and collision data immediately when the level was loaded. While acceptable for small prototypes, this approach scales poorly for larger worlds where only a small percentage of assets are visible or interactable at any given moment.

Loading complete actors unnecessarily increases startup time, memory usage, and asset residency, making it difficult to scale towards an open-world or streaming-based architecture.

The Solution :

To reduce unnecessary memory consumption, I redesigned the asset loading pipeline around Soft Object References.

Instead of treating actors as fully initialized objects, gameplay objects initially exist only as lightweight logical entities. During construction, mesh references are cleared, leaving behind only the gameplay logic contained within the parent actor classes.

Once the player enters an activation range, the framework asynchronously loads only the required mesh assets before enabling rendering, collision, and physics. Until then, actors occupy only the memory required for their gameplay behaviour.

The same architecture is used for weapon pickups. Rather than directly assigning weapon meshes to pickup actors, pickups simply identify which weapon should be spawned. The responsibility of asynchronously loading the appropriate mesh is delegated to the player's weapon component, allowing pickups to remain lightweight while centralizing weapon asset management.

Engineering Decisions :

- Soft Object References
- Asynchronous asset loading
- Lightweight logical actors
- Delayed mesh initialization
- Centralised weapon asset management
- Reduced asset residency
- Separation of gameplay logic from visual assets
- Saleable streaming architecture

Improvements observed :

The most noticeable improvement was a significant reduction in the memory footprint of inactive gameplay objects. Parent gameplay classes typically occupy less than 200 KB, allowing them to remain resident in memory while expensive rendering assets such as meshes, textures, materials, and physics assets are loaded only when required.

This also improves object casting performance since gameplay systems interact primarily with lightweight parent classes rather than fully initialized actors. Because gameplay behaviour remains available independently of rendering assets, systems such as interaction,

inventory, and AI can continue operating without requiring the visual representation to be loaded.

Although currently implemented for weapon meshes and pickups during prototyping, the framework was intentionally designed to scale towards streaming larger categories of assets in future open-world environments.

Learning :

Developing this framework changed how I think about gameplay objects. Rather than viewing an actor as a single object containing both behaviour and presentation, I began treating them as two separate concerns:

- Gameplay logic that should remain lightweight and always available.
- Visual assets that should be streamed only when needed.

This separation improves scalability, reduces memory usage, and provides a strong architectural foundation for larger worlds where thousands of objects may exist logically without all of them needing to be fully loaded at the same time.

Janus - modular interaction framework (UE5)

Currently in development

The Problem :

Gameplay interactions are one of the most frequently repeated systems in games. Traditional implementations often result in every interactable object maintaining its own interaction logic, asset references, UI prompts, animations, and gameplay behaviour. As projects grow, this leads to duplicated code, inconsistent behaviour between interactable objects, and increasing maintenance costs whenever a new interaction type is introduced.

Additionally, interactable objects frequently load all of their visual assets during level initialization, even when they are far outside the player's interaction range, unnecessarily increasing memory usage.

The Solution :

Janus introduces a modular interaction architecture built around Unreal Engine's Actor Components, Data Tables, and Soft Object References.
Every interactable object derives its interaction behaviour from a common *BPC_Interact* component, while specialize interaction components inherit and extend this base functionality for different gameplay scenarios such as player pickups, NPC conversations, world interactions, and object activation.

Rather than hardcoding interaction behaviour, Janus stores interaction definitions inside Unreal Engine Data Tables. Each interaction entry specifies its interaction type using enums (Interact, Talk, Pickup, Open, Close, None, etc.) together with any associated gameplay data and soft references required by that interaction.

Because all visual assets are referenced through Soft Object References, Atlas integrates directly with my asynchronous asset streaming framework. Gameplay objects remain lightweight until the player enters an interaction range, at which point only the required meshes and assets are loaded asynchronously into memory.

Gameplay systems responding to completed interactions communicate through my Hermes event framework, allowing UI, audio, AI, inventory, quests, and gameplay systems to react independently without introducing direct dependencies.

Engineering Decisions :

- Composition using Actor Components (BPC_Interact)
- Data-driven interaction definitions
- Enum-based interaction state machine
- Soft Object References
- Asynchronous asset streaming
- Publish–Subscribe gameplay events (Game Messages subsystem : Epic Games)
- Separation of gameplay logic from visual assets
- Modular inheritance hierarchy

Improvements observed :

The framework significantly reduces duplicated interaction code by centralizing common behaviour inside reusable interaction components. New interaction types can be introduced by extending the base component and adding corresponding Data Table entries rather than modifying existing gameplay systems.

Using Soft Object References also reduces the memory footprint of inactive interactable objects, allowing gameplay behaviour to exist independently of expensive rendering assets until they are actually required.

The modular architecture allows gameplay programmers and designers to extend interaction behaviour with minimal changes to existing code while maintaining a consistent interaction workflow across different object types.

Learning :

Developing Janus reinforced the importance of composition over inheritance and data-driven gameplay design. Rather than treating every interactable object as a unique gameplay implementation, I began viewing interactions as reusable behaviours that could be attached, configured, and extended independently of the objects themselves.

Combining Actor Components, Data Tables, asynchronous asset streaming, and event-driven communication produced a flexible interaction architecture where gameplay systems remain loosely coupled, memory efficient, and significantly easier to extend as project complexity increases.

Atlas - Open World Streaming Framework (unity)

The Problem :

Large open-world games often contain thousands of objects, terrain sections, buildings, NPCs, and gameplay systems. Loading the entire world at once quickly exceeds available GPU and system memory while significantly increasing loading times.

For large environments, only a small portion of the world is actually relevant to the player at any given moment. Loading distant regions wastes both memory and rendering resources, making it difficult to support large worlds on consumer hardware.

The Solution :

Atlas is a chunk-based asynchronous world streaming framework built on top of Unity's Addressables system. The world is divided into 1000 × 1000 metre streaming regions, with each

region containing its own environment, gameplay objects, and world content. During gameplay, Atlas continuously monitors the player's position and asynchronously loads only the 3 × 3 neighbourhood of chunks surrounding the player while unloading distant regions that are no longer required.

Because streaming is performed asynchronously through Addressables, world transitions occur without blocking the main thread, allowing the player to move seamlessly through large environments while maintaining a predictable memory footprint.

Atlas was specifically designed for open-world games using fully dynamic lighting, eliminating the requirement for baked lighting data and enabling large streaming environments without static lightmap dependencies, This system relies on Adaptive light probe volume and light scenarios for global illumination.

Engineering Decisions :

- Chunk-based world partitioning
- 1000 × 1000 metre streaming regions
- 3 × 3 active chunk neighbourhood
- Unity Addressables
- Asynchronous scene loading
- Automatic chunk loading and unloading
- Dynamic lighting support using unity’s adaptive light probe volumes
- Memory-conscious world management

Improvements observed :

Atlas was tested using the complete Grand Theft Auto: San Andreas map extracted into Unity HDRP. Despite the size of the environment, only nearby regions remained resident in memory at any given time.

On a machine equipped with **4 GB of VRAM**, the framework maintained approximately 200 FPS inside the Unity HDRP Editor, demonstrating that aggressive world streaming can dramatically reduce memory requirements while maintaining smooth traversal through large environments.

Because only nearby chunks remain loaded, the framework scales significantly better than traditional scene-based loading approaches and provides a strong foundation for future open-world projects.

Learning :

Developing Atlas fundamentally changed how I think about world representation. Rather than viewing a game world as a single scene, I began treating it as a collection of independently streamable regions whose lifetime should be determined entirely by player proximity.

This project reinforced the importance of asynchronous programming, memory management, and data partitioning when designing systems intended to scale beyond small prototype environments. It also demonstrated that careful world partitioning can have a much greater impact on runtime performance than attempting to optimize rendering alone.

GOAP - Goal Oriented Action Planning (unity)

Repo available : Github

The Problem :

Traditional NPC behaviour in games is often implemented using Finite State Machines (FSMs), where every possible behaviour transition must be manually designed by the programmer. While effective for simple AI, the number of transitions grows rapidly as more behaviours are introduced, making the system increasingly difficult to maintain and extend.

For complex NPC behaviour requiring decision making based on changing world conditions, a more flexible planning system was required.

The Solution :

I implemented a Goal-Oriented Action Planning (GOAP) framework in Unity where NPCs select actions dynamically based on goals, world state, and action preconditions rather than following predefined state transitions.

Actions define their own preconditions and effects, while the planner evaluates which sequence of actions best satisfies the desired goal. This allows new behaviours to be introduced by simply creating additional actions without rewriting the planner itself.

The framework was designed as an educational implementation and was later contributed as an open-source example on GitHub.

Engineering Decisions :

- Goal-Oriented Action Planning
- World State representation
- Action Preconditions
- Action Effects
- Dynamic action planning
- Modular action definitions
- Extensible planner architecture

Improvements needed :

Currently this framework is single threaded and runs on main thread of the game, yes it makes NPCs smart but having a lot of them at once will absolutely tank the frame rate. I should adopt this architecture to burst as planning is almost similar operation across all NPCs. The way I implemented in my game is similar to Alien isolation, one master GOAP is watching everything about player and is controlling a puppet NPC trying to stop player from reaching his goal.

Learning :

Building GOAP introduced me to utility-based AI and planning algorithms used in modern games. It fundamentally changed how I viewed AI behaviour by shifting the focus from manually scripting behaviour transitions towards allowing agents to reason about their objectives based on the current world state.

Freedom Fighters - (Gameplay recreation unity)

Repo available : Github

The Problem :

As I progressed as a gameplay programmer, I wanted to better understand how classic third-person shooters were architected beyond surface-level mechanics. Rather than building isolated gameplay prototypes, I wanted to recreate interconnected gameplay systems inside a complete project.

The Solution :

Freedom Fighters is an ongoing gameplay recreation project focused on rebuilding core gameplay systems using modern Unity architecture.

Current systems include:

- Character Controller
- Inventory Framework
- Weapon Handling
- Equipment Management

The project serves as a sandbox for experimenting with reusable gameplay architecture rather than reproducing the original game exactly. The implementation is publicly available on GitHub.

Engineering Decisions :

- Component-based architecture
- Inventory abstraction
- Weapon framework
- Data-driven equipment
- Reusable gameplay systems
- Modular player controller

Learning :

Freedom Fighters became my personal gameplay laboratory. Rather than implementing isolated mechanics, it taught me how multiple gameplay systems must coexist, communicate, and remain maintainable as project complexity increases. It also strengthened my understanding of gameplay architecture by encouraging reusable systems instead of one-off implementations.

Gargantua - (Volumetric blackhole rendering in HDRP)

Video available here : Youtube

The Problem :

Traditional black hole effects in games are typically achieved using particle systems, meshes, or post-processing effects. While these approaches can create visually convincing results, they lack physically-inspired behaviour such as gravitational lensing, volumetric accretion discs, and dynamic light bending.

I wanted to explore whether a black hole could be rendered procedurally using signed distance fields (SDFs), ray marching, and concepts inspired by general relativity while remaining fully integrated within Unity HDRP.

Big inspiration from the movie Interstellar

The Solution :

The renderer was implemented as a Unity HDRP Custom Pass, allowing the black hole to be rendered independently of the standard rendering pipeline. Rather than modelling the black hole using meshes, the entire effect is generated procedurally through Signed Distance Fields (SDFs). A sphere represents the event horizon while a cylinder is modified by subtracting its central region to create the accretion disc geometry. These implicit surfaces are evaluated through ray marching, allowing the renderer to determine surface intersections and lighting entirely inside the shader.

To simulate gravitational lensing, the viewing rays are warped using mathematical approximations inspired by Einstein's theory of relativity, producing the characteristic bending of light around the event horizon. The accretion disc itself is rendered volumetrically by adapting techniques originally developed for my volumetric cloud renderer, allowing the surrounding gas to exhibit depth and density rather than appearing as a simple textured ring.

Engineering Decisions :

- Unity HDRP Custom Pass
- Signed Distance Fields (SDF)
- Ray Marching
- Procedural Geometry
- Volumetric Rendering
- Relativistic Light Distortion
- Shader-based Lighting
- Reusable Volumetric Cloud Techniques

Improvements observed :

Using implicit geometry eliminated the need for complex meshes while allowing the entire black hole to be generated procedurally inside the shader.

Because the renderer is based on mathematical distance fields rather than traditional geometry, visual characteristics such as the event horizon, disc thickness, and light distortion can be adjusted entirely through shader parameters without modifying any meshes.

Reusing my volumetric cloud rendering techniques for the accretion disc also demonstrated that rendering techniques developed for one problem could be adapted to solve entirely different visual effects.

Learning :

Developing this renderer significantly improved my understanding of procedural rendering and mathematical graphics programming.

Rather than viewing shaders purely as material definitions, I began treating them as mathematical programs capable of generating complete three-dimensional phenomena from equations alone. Implementing signed distance fields, ray marching, volumetric rendering, and relativistic light distortion reinforced how mathematical models can be translated into visually compelling real-time graphics.

The project also demonstrated the importance of building reusable rendering techniques, as the volumetric cloud implementation became the foundation for rendering the accretion disc surrounding the black hole.

This approach is limited to platforms with really good GPUs as raymarching is really expensive effect to render.

Graphics programming experiments

Technologies Explored

Throughout these experiments I worked extensively with Unity's Scriptable Render Pipeline, including HDRP Custom Passes, HDRP Full Screen Passes, URP Scriptable Render Features, Compute Shaders, Shader Graph, Render Textures, ray marching, Signed Distance Fields, volumetric rendering, procedural texture generation, Gaussian filtering, triplanar mapping, and GPU-based simulations.

Compute Shader Experiments

To better understand GPU programming beyond traditional fragment shaders, I implemented several compute shader based experiments focusing on parallel computation and runtime data generation.

The collection includes Conway's Game of Life, where cellular automata are simulated entirely on the GPU, allowing thousands of cells to update simultaneously every frame. I also developed a runtime graffiti and texture painting system for PC VR, where brush strokes are written directly into GPU render textures without modifying underlying mesh geometry. These projects significantly improved my understanding of GPU memory, thread groups, texture read/write operations, and massively parallel algorithms.

Screen Space Rendering

Several experiments focus on post-processing and full-screen rendering techniques using Unity's Scriptable Render Pipeline.

A Death Stranding inspired scanner effect was implemented using URP Scriptable Render Features, generating a world-space scanning wave capable of propagating through scene geometry. I also recreated the characteristic Need for Speed: Most Wanted (2005) speed effect using HDRP Full Screen Passes and Gaussian filtering to produce a radial motion blur effect driven entirely on the GPU.

Another experiment explored dynamic rain and full-screen wetness, where HDRP Full Screen Passes modify surface normals and project rain droplets and water streaks using triplanar mapping, allowing convincing wet surfaces without requiring additional authoring for every individual material.

Procedural Rendering

A significant portion of my rendering experiments focused on procedural graphics generation.

This includes the development of a volumetric black hole renderer built using Signed Distance Fields, ray marching, and mathematical approximations inspired by gravitational lensing. The surrounding accretion disc was generated by adapting techniques originally developed for my volumetric cloud renderer, demonstrating how reusable rendering techniques can be applied across different visual effects.

Learning

This collection fundamentally changed the way I think about graphics programming. Rather than viewing shaders as isolated material definitions, I began treating the GPU as a programmable parallel processor capable of simulation, procedural generation, post-processing, and complete rendering pipelines.

These experiments strengthened my understanding of Unity's rendering architecture while providing practical experience with GPU programming, Scriptable Render Pipelines, procedural rendering, compute shaders, volumetric effects, and real-time optimisation techniques. More importantly, they encouraged experimentation and helped build confidence when approaching unfamiliar rendering problems from first principles.

Leonids - High performance data driven ballistics system UnrealEngine

Video available here : **STILL WORKING ON VIDEO AS OF TODAY**

The Problem :

Traditional projectile implementations in Unreal Engine commonly represent each projectile as an individual Actor or object with its own transform, collision state, movement logic, and gameplay behavior.

While pooling can reduce allocation and spawning overhead, large numbers of active projectiles can still create substantial CPU work because each projectile requires simulation and collision processing every frame.

For projectile-heavy combat, particularly weapons capable of producing large numbers of simultaneous rounds, the objective was to separate **projectile simulation from the Unreal Actor lifecycle** while retaining reliable collision, gameplay events, audiovisual feedback, and integration with Blueprint-driven weapon

The Solution :

Instead of representing every projectile as an Unreal Actor, the system treats projectiles as **data managed centrally by a *UWorldSubsystem***.

The subsystem maintains projectile state using compact parallel arrays containing position, velocity, damage, lifetime, and rendering indices.

Projectile simulation is performed centrally through the subsystem's tick function, allowing weapons and other gameplay systems to submit projectile spawn requests through a Blueprint-callable interface rather than owning individual projectile simulation objects.

Each simulation frame is divided into several stages:

1. Projectile Simulation

For every active projectile:

- Advance its position using its current velocity.
- Apply gravitational acceleration.
- Track projectile lifetime.
- Skip inactive projectiles.

The system uses explicit ballistic integration for the frame, including the gravity term:

$$displacement = velocity \times \Delta t + \tfrac{1}{2} \times gravity \times \Delta t^2$$

This means collision prediction accounts for the projectile's **curved ballistic trajectory**, rather than simply checking the projectile after it has already moved.

2. Predictive Collision Detection

Instead of moving the projectile first and then performing a collision test, the system performs a swept collision query from the current position toward its predicted position.

This provides continuous collision detection for fast-moving projectiles and reduces the possibility of tunnelling through geometry.

3. Parallel Projectile Processing

Active projectiles are processed using Unreal's parallel execution facilities.

Each worker operates on projectile data while recording collision, retirement, and fly-by information into its own task context. Gameplay-affecting operations are deferred until the parallel simulation has completed.

This separates:

Parallel simulation
↓
Result collection
↓
Game-thread dispatch

and avoids directly performing unsafe shared gameplay operations from worker threads.

4. Result Dispatch

Projectile hits are collected and processed after the parallel simulation stage.

The resulting events can then trigger gameplay consequences such as:

- damage,
- projectile retirement,
- impact effects,
- impact sounds,
- decals,
- and other gameplay responses.

This keeps the simulation layer separated from the systems that consume projectile events.

5. Niagara Visualization

Projectile visualization is separated from projectile simulation.

Rather than spawning a Niagara system for every bullet, the subsystem maintains pooled Niagara indices and writes projectile position and velocity information through a **Niagara Data Channel**.

When a projectile is retired, its Niagara slot can be returned to the free-index pool for reuse.

This allows:

CPU projectile simulation
↓
Niagara Data Channel
↓
GPU projectile visualization

without requiring an individual Unreal Actor or Niagara component per projectile.

6. Fly-by Detection

The system also supports projectile fly-by events.

During simulation, nearby components carrying the BulletFlyby tag can be detected and their location and projectile direction recorded. These results are dispatched after the parallel simulation, allowing audio and other feedback systems to react without coupling them directly to the simulation workers.

Key Engineering Decisions

- Centralized projectile simulation using Unreal UWorldSubsystem
- Data-oriented projectile state using compact parallel arrays
- Parallelized projectile simulation
- Predictive swept collision using ballistic displacement
- Gravity-aware projectile trajectories
- Deferred game-thread processing for gameplay side effects
- Niagara Data Channel integration
- Pooled Niagara visualization indices
- Event-driven projectile feedback
- Fly-by detection and directional projectile information
- Actor-independent projectile simulation
- Stress-tested at ~50,000 projectile spawns/sec
- I had limited this project with an Array Of Structures implementation as there is very little difference in unreal engine as of compared to unity’s side of implementation.

Custom Real-Time Rendering Engine

C++ / HLSL — Independent Graphics Programming Project
Video available here : Youtube
Repo available : Github

About the project

A custom real-time rendering engine developed from first principles to understand the architecture and mathematics behind modern game rendering pipelines rather than relying entirely on an existing engine's abstractions.

The project began with rendering a basic triangle and progressively evolved into a complete real-time renderer supporting dynamic scene manipulation, asset importing, physically based rendering, render queues, a render graph, dynamic lighting, shadow mapping, GPU filtering, and skybox rendering.

Development flow

Triangle Rasterization
↓
Dynamic Cube & Scene Manipulation
↓
Texture Mapping
↓
Assimp Asset Importing
↓
Custom Transform System
↓
First-Person Scene Controller
↓
Lighting & Lambertian Shading
↓
GLTF Analysis & PBR Metallic-Roughness
↓
Alpha Blending
↓
Render Queue Architecture
↓
Render Graph
↓
Shadow Mapping
↓
PCF & Slope-Scaled Bias
↓
Point-Light Cubemap Shadows
↓
Skybox & Final Renderer

Engineering Decisions

- Built the rendering pipeline independently instead of depending on an existing game-engine rendering abstraction, requiring direct management of rendering passes, GPU resources, shaders, transforms, and scene state.
- Replaced a sequential collection of rendering operations with a **Render Graph** responsible for coordinating frame clearing, shadow rendering, lighting, object highlighting, and composition.
- Implemented a reusable **Transform system** supporting position, Euler/quaternion rotation, local/world-space transformations, and inverse transformations.
- Implemented **PBR metallic-roughness shading** based on GLTF material data to reproduce physically based material behaviour.

- Implemented shadow mapping for multiple light types, including **cubemap-based point-light shadows**.
- Added **hardware Percentage Closer Filtering (PCF)** and slope-scaled depth bias to improve shadow quality while reducing shadow acne and Peter Panning.
- Built debugging and visualization tools for cameras, projection frustums, scene objects, lighting, and runtime performance.

Learning

The primary goal of the project was not simply to reproduce rendering features, but to understand the abstractions normally hidden by modern game engines.

Working from a triangle upward provided a practical understanding of how scene data eventually becomes pixels on the screen—from transforms and camera projections to GPU buffers, rasterization, lighting, shadow maps, filtering, and final composition.

The project also reinforced an important engineering principle: **the more of the underlying system you understand, the less dependent you become on the abstraction built on top of it.**

This experience significantly strengthened my understanding of real-time graphics, GPU programming, rendering architecture, shader development, and the engineering trade-offs involved in building a rendering pipeline.

Professional Experience

Mad VR Solutions Pvt. Ltd.

Team Lead – R&D & Lead Unity and VR Developer | Remote | **June 2024 – August 2025**
(Company went bankrupt and got dissolved)

- Led the development of high-performance VR surgical simulators, improving real-time physics, interaction fidelity, and rendering efficiency.
- Designed and built scalable XR frameworks, enabling rapid prototyping for new simulators.
- Spearheaded performance optimization, ensuring Meta Quest store compliance and achieving 72+ FPS on standalone VR hardware.
- Developed VR hand tracking & physics-based interactions using Photon Fusion2 for multiplayer collaboration.
- Engineered custom shaders (HLSL) for advanced VR rendering, including screen-space effects, inverted hull outlines, and triplanar textures.
- Trained and mentored junior developers in VR fundamentals, rendering pipelines, and networking solutions.
- Provided technical guidance for integrating new VR hardware and motion-tracking systems.

One Immersive Pvt. Ltd.

Senior Unity Developer & XR R&D engineer | Hyderabad, India | **Nov 2023 – June 2024**

- Developed AR-based outdoor navigation using Mapbox VR for virtual real estate tours, enabling video streaming at landmarks.
- Built indoor AR navigation systems using Microsoft Spatial Anchors & Azure DB for precise indoor positioning.
- Designed a Geospatial AR Quest system, displaying AR banners and interactive content at significant city landmarks.
- Created a VR capture system similar to NVIDIA Ansel, using compute shaders to stitch stereo VR images inside Unity.

Turing – Crew Crafts Media

Unity Developer | Remote (California, USA) | **Aug 2022 – Aug 2023**
Project: Dance Kitchen (Retired app on Play Store) (Available on app store)

- Led the development of Dance Kitchen, an interactive dance training app teaching Salsa, Bachata, and Mambo.
- Designed a dynamic content system using Timeline & Custom Data Events, enabling real-time updates via Digital Ocean.
- Engineered custom UI shaders for enhanced visuals and rendering performance.
- Built saleable footwork and body weight shift mechanics, allowing seamless instructor-led guidance in VR.
- Developed and optimized real-time audio synchronization with character animations for immersive dance lessons.
- Managed the full app development life-cycle, from UI programming to back-end integration and optimization.

IACG Multimedia

Unity Developer & Game Dev Professor | Hyderabad, Telangana | **Oct 2021 – Dec 2023**

- Developed custom shaders & rendering solutions, including a Triplanar Shader for far LOD objects, optimizing texture performance.
- Engineered IK systems using Unity's Animation Rigging, collaborating closely with the 3D team for character animation.
- Taught advanced game development skills to students, covering modular systems, Unity’s lightmap switching, and optimization techniques.
- Mentored teams for Dancing Atoms’ WCP Program, ensuring technical excellence in their projects.
- Worked on outsourced projects for Go Live Gaming, 5th Ocean Entertainment, and 7 Seas Entertainment.
- Introduced Utility AI & GOAP for complex NPC decision-making systems.

Research & Collaboration

- XR Sandbox with INTI University Malaysian collaboration
- Part time game dev tutor at VR Siddhartha

Recognition and Awards

News paper appearance on 12th February 2026 for hosting entrepreneurship event in Chirala, St. Ann's engineering college.

ఆంధ్రపత్రిక
12/02/2026 | Page No: 8
andhrapatrikaa.com

యువతలో సృజనాత్మకశక్తిని వెలికితీసేందుకే ఐడియాధాన్: కె. జగదీష్ బాబు

వేటపాలెం, ఫిబ్రవరి 11 (ఆంధ్రపత్రిక): యువతలో సృజనాత్మకశక్తిని పెంపొందించి, నూతన ఇన్నోవేటివ్ ఐడియాస్ ను వెలికితీసేందుకు ఐడియాధాన్ పై శిక్షణ విద్యార్థులకు ఉపయుక్తకరమని సెయింట్ అన్స్ కాలేజ్ అఫ్ ఇంజనీరింగ్ అండ్ టెక్నాలజి చీరాల ప్రిన్సిపాల్ డాక్టర్ కె. జగదీష్‌బాబు తెలిపారు. బుధవారం కళాశాలలోని ఎలక్ట్రానిక్స్ అండ్ కమ్యూనికేషన్ ఇంజనీరింగ్ విభాగము వారి ఆధ్వర్యములో 3 రోజుల పాటు బి.టెక్ 2వ సంవత్సరము చదువుతున్న విద్యార్థులకు ఐడియా ధాన్ శిక్షణ కార్యక్రమం ప్రారంభించినట్లు తెలిపారు. ఈ వర్క్ షాపును విజయవాడ కు చెందిన ఇన్నోవర్సిటి సహకారముతో టెక్నికల్ వర్సన్స్ అర్. గోపికృష్ణ, వాహబ్, ఎ. హేమంత్, ఎ. హర్షవర్ధన్ లు అధ్వర్యంలో నిర్వహిస్తున్నట్లు తెలిపారు. అనంతరం ఈసీఈ విభాగాధిపతి డాక్టర్ డి. రాజేంద్ర ప్రసాద్ మాట్లాడుతూ అధునిక టెక్నాలజి ప్రపంచములో ఇన్నోవేటివ్ ఐడియాస్ ఎంతో ప్రాధాన్యత వహిస్తున్నదని టెక్నాలజిని ఉపయోగిస్తున్న ప్రతిఒక్కరు ఆశావహ దృక్పదాన్ని కలిగి ఉండాలని, తమ అలోచనలను అధునిక టెక్నాలజిలను ఉపయోగించి అందరికి అందుబాటులోకి తెవాలన్నారు.

సూర్య — గురువారం 12 ఫిబ్రవరి 2026

సెయింట్ ఆన్స్ ఇంజినీరింగ్ కళాశాలలో మూడు రోజుల ఐడియాతాన్ ప్రారంభం

సూర్య వేటపాలెం ప్రతినిధి ఏ అనిల్ నాయుడు :

చీరాల పట్టణంలోని సెయింట్ అన్స్ కాలేజ్ ఆఫ్ ఇంజినీరింగ్ అండ్ టెక్నాలజిలో మూడు రోజుల పాటు నిర్వహించనున్న ఐడియాతాన్ కార్యక్రమం ప్రారంభమైందని కళాశాల సెక్రటరి వసుమా రామకృష్ణా రావు కరస్పాండెంట్ శ్రీమంతుల లక్ష్మణ రావు సంయుక్తంగా ఒక ప్రకటనలో తెలిపారు. ఈ ఐడియాతాన్‌ను ఎలక్ట్రానిక్స్ అండ్ కమ్యూనికేషన్ ఇంజినీరింగ్ విభాగం ఆధ్వర్యంలో, బి.టెక్ ద్వితీయ సంవత్సరం విద్యార్థుల కోసం మూడు రోజుల పాటు నిర్వహిస్తున్నట్లు కళాశాల ప్రిన్సిపాల్ డా॥ కె. జగదీష్ బాబు తెలిపారు. ఈ వర్క్‌షాప్‌ను ఇన్నోవర్సిటీ, విజయవాడ వారి సహకారంతో నిర్వహిస్తున్నట్లు ఈ సి ఈ విభాగాధిపతి డా॥ డి. రాజేంద్ర ప్రసాద్ పేర్కొన్నారు. ఈ కార్యక్రమంలో ఇన్నోవర్సిటీ నుంచి ఆర్. గోపికృష్ణ, వాహబ్, ఎ. హేమంత్, ఎ. హర్షవర్ధన్ పాల్గొంటున్నారని తెలిపారు. ఐడియాతాన్‌ను ప్రారంభిస్తూ కళాశాల ప్రిన్సిపాల్ మాట్లాడుతూ, అధునిక టెక్నాలజీ ప్రపంచంలో ఇన్నోవేటివ్ ఐడియాలకు అత్యంత ప్రాధాన్యత ఉందని, టెక్నాలజీని వినియోగిస్తున్న ప్రతి ఒక్కరూ అశావహ దృక్పథాన్ని కలిగి ఉండాలని సూచించారు. విద్యార్థులు తమ ఆలోచనలను ఆధునిక సాంకేతికతను ఉపయోగించి సమాజానికి ఉపయోగపడే విధంగా అందుబాటులోకి తీసుకురావాలని ఆకాంక్షించారు. అలాగే విద్యార్థులు ఆధునిక టెక్నాలజీలు సాఫ్ట్‌వేర్‌లపై అవగాహన పెంపొందించుకుని ఉన్నత ఉద్యోగ అవకాశాలు సాధించాలని కోరారు. ఈ కార్యక్రమంలో డా॥ సి. సుబ్బారావు (డైరెక్టర్ -- అక్రిడిటేషన్స్), వి. దొర నాగ సాయినాద్ (డైరెక్టర్ -- అడ్మినిస్ట్రేషన్), వివిధ విభాగాధిపతులు, అధ్యాపకులు, విద్యార్థినీ విద్యార్థులు పాల్గొన్నారు.

Hosted a game development workshop on unity and participated as the tutor at SRM AP January 2026.

Hosted a game development workshop on unity and participated as the tutor at VR Siddhartha university July 2025.